System Verification and Validation Plan for Software Engineering

Team 18, Team Magic
Allen Tong
Amos Edeaghe
Jacob Brodersen
Joshua Engel
Nehan Munir

November 3, 2026

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 1
- 2.4 Relevant Documentation . . . 2

3 Plan **2**
- 3.1 Verification and Validation Team . . . 2
- 3.2 SRS Verification . . . 2
- 3.3 Design Verification . . . 2
- 3.4 Verification and Validation Plan Verification . . . 3
- 3.5 Implementation Verification . . . 3
- 3.6 Automated Testing and Verification Tools . . . 3
- 3.7 Software Validation . . . 4

4 System Tests **4**
- 4.1 Tests for Functional Requirements . . . 4
  - 4.1.1 Round Setup . . . 4
  - 4.1.2 Tasks and Answer Checking . . . 6
  - 4.1.3 Scoring . . . 10
  - 4.1.4 Saving and Recovery . . . 11
  - 4.1.5 3D Visualization . . . 12
  - 4.1.6 Guided Learning and Tutorial . . . 13
  - 4.1.7 Accounts and Competitive Play . . . 14
  - 4.1.8 Wellbeing . . . 18
- 4.2 Tests for Nonfunctional Requirements . . . 18
  - 4.2.1 Area of Testing1 . . . 19
  - 4.2.2 Area of Testing2 . . . 19
- 4.3 Traceability Between Test Cases and Requirements . . . 19

5 Unit Test Description **21**
- 5.1 Unit Testing Scope . . . 22
- 5.2 Tests for Functional Requirements . . . 22
  - 5.2.1 Module 1 . . . 22

5.2.2 Module 2 23
5.3 Tests for Nonfunctional Requirements 23
5.3.1 Module ? 23
5.3.2 Module ? 24
5.4 Traceability Between Test Cases and Modules 24

6 Appendix **25**
6.1 Symbolic Parameters 25
6.2 Usability Survey Questions? 25

7 Appendix — Generative AI Use Disclosure **26**
7.1 AI Tools Used 26
7.2 How and Where Used 26
7.3 Verification of AI-Generated Content 26

8 Appendix — Reflection **27**

List of Tables

1 Traceability Between Functional Requirements and System Tests 21
[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

[symbols, abbreviations, or acronyms — you can simply reference the SRS (Author, 2019) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

2.1 Summary

[Say what software is being tested. Give its name and a brief overview of its general functions. —SS]

2.2 Objectives

[State what is intended to be accomplished. The objective will be around the qualities that are most important for your project. You might have something like: "build confidence in the software correctness," "demonstrate adequate usability." etc. You won't list all of the qualities, just those that are most important. —SS]

[You should also list the objectives that are out of scope. You don't have the resources to do everything, so what will you be leaving out. For instance, if you are not going to verify the quality of usability, state this. It is also worthwhile to justify why the objectives are left out. —SS]

[The objectives are important because they highlight that you are aware of limitations in your resources for verification and validation. You can't do everything, so what are you going to prioritize? As an example, if your system depends on an external library, you can explicitly state that you will assume that external library has already been verified by its implementation team. —SS]

2.3 Custom Documents

[Summarize the CDs (Custom Documents) that were tackled by this project. CDs can include usability testing, code walkthroughs, user documentation, formal proof, GenderMag personas, Design Thinking, etc. CDs should have already been approved by the course instructor as included in your problem statement. You can use a pull request to update your CDs (in TeamComposition.csv or Repos.csv) if your plan changes as a result of the VnV planning exercise. —SS]

2.4 Relevant Documentation

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your documents and within those entries include a hyperlink to the documents. —SS]

Author (2019)

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

3 Plan

[Introduce this section. You can provide a roadmap of the sections to come. —SS]

3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the review. Will you have a meeting? What will you present? What questions will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

3.3 Design Verification

[Plans for design verification —SS]

[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

3.4 Verification and Validation Plan Verification

[The verification and validation plan is an artifact that should also be verified. Techniques for this include review and mutation testing. —SS]
[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

3.5 Implementation Verification

[You should at least point to the tests listed in this document and the unit testing plan. —SS]

[In this section you would also give any details of any plans for static verification of the implementation. Potential techniques include code walkthroughs, code inspection, static analyzers, etc. —SS]

[The final class presentation in CAS 741 could be used as a code walkthrough. There is also a possibility of using the final presentation (in CAS741) for a partial usability survey. —SS]

3.6 Automated Testing and Verification Tools

[What tools are you using for automated testing. Likely a unit testing framework and maybe a profiling tool, like ValGrind. Other possible tools include a static analyzer, make, continuous integration tools, test coverage tools, etc. Explain your plans for summarizing code coverage metrics. Linters are another important class of tools. For the programming language you select, you should look at the available linters. There may also be tools that verify that coding standards have been respected, like flake9 for Python. —SS]

[If you have already done this in the development plan, you can point to that document. —SS]

[The details of this section will likely evolve as you get closer to the implementation. —SS]

3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

4 System Tests

4.1 Tests for Functional Requirements

The tests in this section verify the functional requirements in Section 6.2 of the SRS. Each test covers a group of related requirements, and the tests are grouped into the same areas as the requirements: round setup, tasks and answer checking, scoring, saving and recovery, 3D visualization, guided learning, accounts and competitive play, and wellbeing. Requirements are referred to by their numbers in Section 6.2 of the SRS (for example, FR9). Section 4.3 shows which tests cover which requirements. Tests marked Automatic are run by a script. Tests marked Manual are performed by a tester who checks the result on screen. Expected values come from Dr. Rapoport's rules documents and from the SRS.

4.1.1 Round Setup

These tests cover FR1 to FR6.

1. ST-FR-1

Control: Automatic

Initial State: The game is loaded and no round is in progress.

Input: The tester presses START and then, without choosing a scoring base, tries to press START again. The tester then chooses the octal base and presses START.

Output: After the first START, 512 cells are displayed with their numbers hidden. The starter and timer do not appear until a base is chosen. After the base is chosen and START is pressed, one starter number is shown in its cell with a white background, and a timer showing elapsed seconds begins counting.

Test Case Derivation: The cube has 8 x 8 x 8 = 512 cells (FR1). A base must be chosen before the starter appears (FR4, FR5).

How test will be performed: A script drives the interface, counts the displayed cells, checks that none shows a number, and reads the starter cell and timer at each step.

2. ST-FR-2

Control: Automatic

Initial State: The game is loaded, with the random number generator replaced by a seeded generator.

Input: 20,000 rounds are started.

Output: All three plane colours (yellow, green, blue) and all 384 orientations occur at least once, and no round uses a plane colour or orientation outside these sets.

Test Case Derivation: FR2 and FR3 require a random choice from 3 colours and 384 orientations. With 20,000 rounds, each orientation is expected about 52 times, so a missing orientation would indicate a defect and not chance.

How test will be performed: A script starts the rounds, records the plane colour and orientation of each, and compares the sets observed against the allowed sets.

3. ST-FR-3

Control: Automatic

Initial State: A new player with an empty used-starter list.

Input: 192 rounds are played, recording the starter of each, followed by a 193rd round.

Output: The 192 starters are all different. After the 192nd round, the list is cleared, so the 193rd starter may repeat an earlier starter.

Test Case Derivation: FR6 says no starter is reused until 192 rounds have been played, then the list is cleared.

How test will be performed: A script plays the rounds in a loop and checks for duplicates among the first 192 starters and for the cleared list afterward.

4.1.2 Tasks and Answer Checking

These tests cover FR7 to FR26 and FR30.

1. ST-FR-4

   Control: Automatic

   Initial State: Task 1 is awaiting a cell selection. The starter is at position (0,0,0).

   Input: The tester selects 20 different incorrect cells, then selects the target cell, then selects the target cell again. In a separate run, the tester clicks a cell while no selection is expected (for example, while the Addition Box is shown).

   Output: Every incorrect cell turns red and the round continues with no limit or lock-out. The target cell turns white. The second selection of the white cell is ignored. The click made while no selection is expected changes nothing.

   Test Case Derivation: FR8 gives white for target cells and red for others, with no limit on incorrect selections, and ignores cells that are already white and selections made while none is expected. The target for task 1 is the cell whose x, y, and z coordinates each differ by 4 modulo 8, which is (4,4,4) (FR9).

How test will be performed: A script clicks each cell and reads its colour after each click.

2. ST-FR-5

   Control: Automatic

   Initial State: A fixed cube array, a fixed plane colour, and a fixed starter for each of the tasks 1 to 8. For tasks 4 to 7, one setup uses a continuous line and another uses a split line.

   Input: For each task, the tester selects every cell of the expected line, in any order. For tasks 4 to 7, this is done once for a continuous line and once for a split line. For task 8, the tester also selects the triagonal used in task 7.

   Output: Each expected target cell turns white and no other cell does. The number of target cells is 1 for task 1, 3 for task 2, and 7 for tasks 3 to 8. Continuous and split lines are both accepted for tasks 4 to 7. In task 8, the triagonal from task 7 is not accepted. Across a round, the tasks appear in order from 1 to 8 and no two tasks use the same target cells.

   Test Case Derivation: The expected cells for each task come from the task definitions (FR9 to FR16) in Dr. Rapoport's rules, worked out by hand for each fixed setup and stored with the test. FR7 requires the eight tasks in order and no repeated target cells.

   How test will be performed: A script loads each fixed setup, selects the stored cells, and checks cell colours and counts. A second script generates 1,000 rounds and compares the target sets of all task pairs in each round.

3. ST-FR-6

   Control: Automatic

   Initial State: A task 2 setup in octal with starter 1 and the first additional number in the line equal to 2. A second setup is at the Subtraction Box for each of the cases in the table below.

   Input: The tester tries to choose 0, 1, 2, and 3 additional numbers for task 2, and 0, 1, 5, and 6 for task 3. The tester then chooses 1 additional

number for task 2 and enters 4 (incorrect) and then 3 (correct) in the Addition Box, pressing DONE each time. At the Subtraction Box, the tester enters a wrong answer and then the correct answer for each case in the table.

Output: 1 to 2 additional numbers are accepted for task 2 and 1 to 5 for tasks 3 to 8, and other values are rejected. After a valid choice, the starter and exactly that many additional numbers are shown in a column. The answer 4 is cleared and the box turns red, and the answer 3 is accepted and the additional number appears in white. A wrong subtraction answer is cleared and the box turns red. The correct answer in each case is accepted, the timer stops, and the remaining cells of the line are shown in white (for task 1, the answer is shown in the complementary cell).

Test Case Derivation: FR17 gives the allowed ranges and FR18 gives the column. $1 + 2 = 3$ in octal (FR19). FR20 gives the constants 1001 for task 1, 2002 for task 2, and 4004 for tasks 3 to 8, and the expected answer is the constant minus the starter or sum, in octal (FR21).

How test will be performed: A script enters each value, reads the box colour, box contents, and cell colours, and compares the accepted subtraction answer with an independent octal subtraction function.

All values in the table below are octal.

Task	**Starter or sum**	**Constant**	**Answer**
1	25	1001	754
2	700	2002	1102
5	1234	4004	2550

4. ST-FR-7

Control: Automatic

Initial State: The Subtraction Box is shown. A round is awaiting a cell selection in a separate run.

Input: The tester presses DONE with an empty box, then enters “8”, “abc”, and “7a”, pressing DONE each time. In the separate run, the

tester presses a key that has no function in the current state (for example, F9).

Output: Each invalid answer shows a message asking for a valid answer and is not counted as an incorrect answer, so the count of incorrect attempts does not change. The unused key shows a message saying the input was not recognized, and the round state and timer do not change.

Test Case Derivation: FR22 allows only the octal digits 0 to 7 and says invalid input must not count as incorrect. FR23 requires a message for unrecognized input.

How test will be performed: A script enters each value or key press and compares the message and the incorrect-attempt counter before and after.

5. ST-FR-8

   Control: Automatic

   Initial State: Task 3 has just been completed, with the white cells and question on screen and the timer at 40 seconds. In separate runs, a round is at the Subtraction Box of task 8, and a round is in progress at task 4 with a logged-in user.

   Input: The tester presses START to begin task 4. In the second run, the tester enters the correct answer to the task 8 subtraction. In the third run, the tester presses STOP GAME.

   Output: The cells of task 3 other than the starter return to their original colour, the previous question and answers are cleared, and the timer continues from 40 seconds. After task 8, the round ends, the final score is displayed, and a new round can be started. After STOP GAME, the round ends, all progress is erased, no score is recorded in statistics or leaderboards, and the game returns to its ready state.

   Test Case Derivation: FR24, FR25, and FR30 specify these outcomes.

   How test will be performed: A script compares cell colours, question text, and timer before and after each action, and checks the statistics and leaderboard records after STOP GAME.

6. ST-FR-9

   Control: Automatic

   Initial State: The cube array is loaded with one value altered, so one pair of cells that should sum to 1001 (octal) does not.

   Input: The tester starts a round that would use the altered pair.

   Output: The system does not present the task, generates a new setup, and writes the failure to the log.

   Test Case Derivation: FR26 requires the sums and expected answers to be checked before each task, with a new setup and a log entry if a check fails.

   How test will be performed: A script alters the array, starts the round, and checks that the task is replaced and that a log entry exists.

4.1.3 Scoring

These tests cover FR27 to FR29.

1. ST-FR-10

   Control: Automatic

   Initial State: A task is complete in the stated scoring base with the stated number of additional numbers, and the time spent on the task is as stated in the table.

   Input: The tester answers the subtraction correctly in each case.

   Output: The task score is as listed in the table.

   Test Case Derivation: FR27 gives the point values: decimal 10, 20, 30, 30, 40, 40, 60, 60 for tasks 1 to 8, and octal 14, 30, 44, 44, 60, 60, 110, 110, which are 12, 24, 36, 36, 48, 48, 72, 72 in decimal. Each additional number adds 10 (decimal) or 14 (octal, 12 in decimal). FR28 gives score = basic score x 1350 / t, rounded to the nearest whole number. For example, task 3 in decimal at 30 s is 30 x 1350 / 30 = 1350. Task 1 at 7 s is 10 x 1350 / 7 = 1928.57, which rounds to 1929.

How test will be performed: A script sets the timer to the stated time, completes each task, and compares the score shown with the value from an independent calculation.

Task	Base	Additional numbers	Time	Task score
1	Decimal	0	7 s	1929
1	Decimal	0	30 s	450
1	Octal	0	30 s	540
2	Decimal	2	30 s	1800
3	Decimal	0	30 s	1350
3	Octal	0	30 s	1620
8	Decimal	0	30 s	2700
8	Octal	0	30 s	3240

2. ST-FR-11

   Control: Automatic

   Initial State: A full round in decimal with a fixed time of 30 seconds per task and no additional numbers.

   Input: The tester completes all eight tasks, giving one incorrect answer along the way.

   Output: The running total after each task equals the sum of the task scores so far. The final score is (10 + 20 + 30 + 30 + 40 + 40 + 60 + 60) x 1350 / 30 = 290 x 45 = 13,050. The incorrect answer does not change the total.

   Test Case Derivation: FR29 says the total is displayed after each task and the final score is the sum of the eight timed scores.

   How test will be performed: A script plays the round and reads the total after each task.

4.1.4 Saving and Recovery

These tests cover FR31 and FR32.

1. ST-FR-12

   Control: Automatic

   Initial State: A round is in progress at task 3 with the timer at 25 seconds.

   Input: The tester (1) reloads the page and chooses to resume. (2) With the save location made to fail, makes a move that changes the round state. (3) Alters a stored saved round, then opens the page.

   Output: (1) The round resumes at task 3 with the same cell colours and the timer at its saved value. (2) The system tries to save 3 times, then tells the user that saving failed, and the round continues. (3) The system detects that the saved round is invalid, discards it, and tells the user.

   Test Case Derivation: FR31 saves the round at every state change, retries up to 3 times, and reports failure. FR32 restores a valid saved round and discards an invalid one.

   How test will be performed: A script plays to this point, reloads the page, and compares the restored state. It then makes the save call fail and counts the attempts and messages, then edits one stored value, reloads, and checks for the message and for an empty saved state.

4.1.5 3D Visualization

These tests cover FR33 to FR37.

1. ST-FR-13

   Control: Manual

   Initial State: A round is in progress at task 2 with a few cells already selected.

   Input: The tester switches from the slice display to the 3D view and back. In the 3D view, the tester rotates and zooms the cube with the mouse, then with the keyboard only. The tester then chooses a layer along each of the three axes, steps to the adjacent layer, and restores all layers.

Output: Each number is in the same cell in both views, selected cells show the same colours, the round state does not change, and the timer keeps running. The cube rotates and zooms with both input methods. Only the chosen layer is shown clearly with the other layers faded or hidden, the chosen layer and the selected cells are highlighted in visibly different ways, stepping moves to the adjacent layer, and restoring shows all layers.

Test Case Derivation: FR33 and FR36 require consistency between the views and no pause in the timer. FR34 requires pointer and keyboard control. FR35 describes the layer controls.

How test will be performed: The tester compares 10 cells chosen across the cube in both views, notes the timer before and after each switch, and repeats the layer steps for each axis.

2. ST-FR-14

   Control: Manual

   Initial State: The 3D view is shown on a device with reduced performance (for example, a browser with its CPU throttled).

   Input: The tester keeps the 3D view open until its performance falls below the minimum level in the Non-Functional Requirements.

   Output: The system warns the user and offers to switch to the slice display. Accepting returns the user to the slice display with the round state unchanged.

   Test Case Derivation: FR37. The minimum level is set in the Non-Functional Requirements.

   How test will be performed: The tester throttles the browser using its developer tools and observes the warning.

4.1.6 Guided Learning and Tutorial

These tests cover FR38 to FR41.

1. ST-FR-15

   Control: Manual

Initial State: The user opens the tutorial.

Input: The tester completes the tutorial, then replays it, then exits it part way through.

Output: The tutorial includes practice for all eight task types, shows only instructions that match the current step, and can be skipped, exited, or replayed at any time. Nothing from the tutorial appears in statistics or leaderboards.

Test Case Derivation: FR38.

How test will be performed: The tester follows the tutorial, checks the instruction for each step, and checks the statistics page afterward.

2. ST-FR-16

   Control: Automatic

   Initial State: Task 3 is awaiting a cell selection.

   Input: The tester selects an incorrect cell, requests a hint, and then tries to request the solution after 0, 2, and 3 incorrect selections. In a separate run, the tester gives 3 incorrect answers to the same addition.

   Output: After the incorrect cell, a message names the type of line being sought and the rule it follows, without naming a cell. The hint states the coordinate step (for example, "each coordinate differs by 4") without identifying a cell, and the request is recorded. The solution can be requested only after 3 incorrect attempts at the same step. For cell selection, the correct cells are highlighted one after another in the order the line is traced. For an addition, the worked octal calculation is shown. Each request is recorded.

   Test Case Derivation: FR39, FR40, and FR41.

   How test will be performed: A script makes the attempts, reads each message, checks that none contains the number or coordinates of a target cell, and checks when the solution option becomes available.

4.1.7 Accounts and Competitive Play

These tests cover FR42 to FR53.

1. ST-FR-17

   Control: Automatic

   Initial State: No user is logged in and network traffic is being recorded. The account creation page is available.

   Input: The tester plays a full round as a guest. The tester then tries each of the account values in the table below.

   Output: The guest round completes and no data about it is sent to the online service. Values marked Accept are accepted after the user confirms the entered username and email address. Values marked Reject are rejected with a message giving the reason.

   Test Case Derivation: FR42 says no guest data is sent online. FR43 requires a unique username, a unique valid email, and a password of at least 8 characters. FR44 requires 3 to 20 characters and no prohibited terms. The boundary values 2, 3, 20, and 21 characters test each limit.

   How test will be performed: A script plays the round and inspects the recorded requests, then submits each account case and reads the response. A test account already registered is used for the duplicate cases.

Case	**Value**	**Expected**
Username, 2 characters	ab	Reject
Username, 3 characters	abc	Accept
Username, 20 characters	20 letters	Accept
Username, 21 characters	21 letters	Reject
Username with prohibited term	a term from the list	Reject
Duplicate username	an existing name	Reject
Email without “@”	user.example.com	Reject
Duplicate email	an existing email	Reject
Password, 7 characters	Abcdef1	Reject
Password, 8 characters	Abcdefg1	Accept

2. ST-FR-18

Control: Manual

Initial State: One registered account with a test email inbox.

Input: The tester logs in with the correct credentials and logs out, then tries a wrong password with a real username and a username that does not exist. The tester then requests a password reset for the registered address and for an address that is not registered.

Output: Correct credentials log the user in and out. Both login failures show the same message, which does not say whether the username or password was wrong. A reset link arrives only for the registered address, and the page shows the same message in both reset cases.

Test Case Derivation: FR45 and FR46.

How test will be performed: The tester submits each case, compares the two failure messages, and checks the inbox and the page text.

3. ST-FR-19

Control: Automatic

Initial State: A logged-in user with no completed rounds. A leaderboard for decimal scoring holds 10 entries.

Input: The user completes two rounds in decimal with final scores 10,000 and 12,000 and times 300 s and 280 s. One round scores higher than the lowest leaderboard entry and one scores lower than all entries. The tester then sends a request directly to the service to add a score of 99,999 without playing a round.

Output: Each completed round is recorded with its score, base, total time, and date. The statistics show 2 completed rounds, a highest score of 12,000, an average of 11,000, and a fastest time of 280 s for decimal, and no data for octal. The higher score appears on the leaderboard within 1 minute and the lowest entry is removed, the lower score does not appear, only username and score or time are shown, and octal and decimal boards are separate. The direct request is rejected and no record is created or changed.

Test Case Derivation: FR47, FR48, and FR49. The average is (10,000 + 12,000) / 2 = 11,000. FR50 allows records to be created only from rounds the system has completed.

How test will be performed: A script plays the rounds with a controlled timer, reads the statistics page, polls the leaderboard, and checks the records before and after the direct request.

4. ST-FR-20

   Control: Automatic

   Initial State: Two different users are logged in on the same day.

   Input: Both open the daily challenge and one completes it. The tester then copies the shared summary. The system date is moved forward one day and the daily challenge is opened again.

   Output: Both users get the same plane colour, orientation, and starter, and a leaderboard of that day's results is available. The summary contains the date, final score, and total time, and no cell number or answer. On the next day the setup is different.

   Test Case Derivation: FR51 and FR52.

   How test will be performed: A script reads the setup for both users, searches the copied text for any cell value or answer from the round, and reads the setup again after the date changes.

5. ST-FR-21

   Control: Automatic

   Initial State: A logged-in user with 3 recorded rounds and entries on the leaderboard.

   Input: The user deletes their account.

   Output: The username, email, credentials, scores, statistics, and leaderboard entries are removed, and logging in again fails.

   Test Case Derivation: FR53.

   How test will be performed: A script deletes the account and then queries each place the data was stored.

4.1.8 Wellbeing

These tests cover FR54 and FR55.

1. ST-FR-22

   Control: Automatic

   Initial State: A session is in progress with a round at task 2 and the timer at 20 seconds. The session clock can be advanced. The 3D view is shown with animations on.

   Input: The tester advances the session clock by one hour and then two hours. The tester then turns animations off, rotates the cube, and selects a cell.

   Output: A reminder showing how long the user has been playing appears after each hour, and the round state and round timer are unchanged. With animations off, the cube rotates without animation and cell feedback has no visual effects, and the change applies immediately with no reload.

   Test Case Derivation: FR54 and FR55.

   How test will be performed: A script advances a mocked clock and reads the reminder text, then compares the round state and timer before and after. The animation check is made by the tester on screen.

4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention reporting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

4.2.1 Area of Testing1

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

4.2.2 Area of Testing2

...

4.3 Traceability Between Test Cases and Requirements

Table 1 shows which system tests in Section 4.1 support each functional requirement in the SRS.

Requirement	**Tests**
FR1	ST-FR-1
FR2	ST-FR-2
FR3	ST-FR-2
FR4	ST-FR-1
FR5	ST-FR-1
FR6	ST-FR-3
FR7	ST-FR-5
FR8	ST-FR-4
FR9	ST-FR-4, ST-FR-5
FR10 to FR16	ST-FR-5
FR17	ST-FR-6
FR18	ST-FR-6
FR19	ST-FR-6
FR20	ST-FR-6
FR21	ST-FR-6
FR22	ST-FR-7
FR23	ST-FR-7
FR24	ST-FR-8
FR25	ST-FR-8
FR26	ST-FR-9
FR27	ST-FR-10
FR28	ST-FR-10
FR29	ST-FR-11
FR30	ST-FR-8
FR31	ST-FR-12
FR32	ST-FR-12
FR33	ST-FR-13
FR34	ST-FR-13
FR35	ST-FR-13

Requirement	Tests
FR36	ST-FR-13
FR37	ST-FR-14
FR38	ST-FR-15
FR39	ST-FR-16
FR40	ST-FR-16
FR41	ST-FR-16
FR42	ST-FR-17
FR43	ST-FR-17
FR44	ST-FR-17
FR45	ST-FR-18
FR46	ST-FR-18
FR47	ST-FR-19
FR48	ST-FR-19
FR49	ST-FR-19
FR50	ST-FR-19
FR51	ST-FR-20
FR52	ST-FR-20
FR53	ST-FR-21
FR54	ST-FR-22
FR55	ST-FR-22

Table 1: Traceability Between Functional Requirements and System Tests

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this

section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

Initial State:

Input/Condition:

Output/Result:

How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

Author Author. System requirements specification. https://github.com/..., 2019.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

Claude Sonnet 5.5 (Medium) by Anthropic

7.2 How and Where Used

Claude was used to help draft the system tests for the functional requirements in Section 4.1. Using the functional requirements from Section 6.2 of the SRS as context, the system tests were created, with related requirements grouped into one test. Claude was also used to build the functional requirements traceability table in Section 4.3.

7.3 Verification of AI-Generated Content

The system tests were checked against the functional requirements in the SRS and the traceability table against the tests. Scoring and octal calculations were recomputed, and one incorrect example was corrected before inclusion. The team remains responsible for the accuracy and completeness of all content in this document, regardless of whether it was generated by AI or written by a team member.

8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Questions 3 and 4 have been answered as a team.

3. Another skill we will need is automated testing of the browser interface, since many of the system tests in Section 4.1 are run by a script that clicks cells, enters answers, and reads the screen.

4. Allen is focusing on learning automated browser interface testing. The first approach is reading the documentation and getting started guides for a browser testing tool such as Playwright. The second approach is to practice by writing a few of the system tests from Section 4.1 (for example, ST-FR-1 and ST-FR-4) against an early version of the game, so the learning is applied directly to our own project.

Allen Tong

1. What went well for me was writing Section 4.1. Since I wrote the functional requirements in the SRS, I already knew how each requirement was supposed to behave, so the tests came naturally from them. I could work out the expected values directly from the rules, such as the scoring formula and the octal subtraction constants, and group related requirements into a single test. It also made the traceability table in Section 4.3 straightforward because I knew which requirements each test needed to cover.

2. One pain point was the number of tests. My first draft had a test for almost every requirement, which came to 46 tests and was too much for this document. I resolved this by grouping related requirements together, which brought it down to 22 tests while still covering all 55 requirements.